Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Отчёт по лабораторной работе №2

Дисциплина: “Вычислительная математика”

Тема: Численное решение нелинейных уравнений и систем

Вариант 10

Выполнил**:** студент группы P3215 Панченко Антон Дмитриевич

Проверил**:** Наумова Надежда Александровна

Дата сдачи**:**

Санкт-Петербург, 2026

Оглавление

1. Цель работы
2. Порядок выполнения работы
3. Рабочие формулы используемых методов.
4. Графики функций на исследуемом интервале.
5. таблицы вычислительной части 1
6. Подробное решение системы нелинейных уравнений
7. Листинг программы
8. Результаты выполнения программы
9. Выводы по работе.

1. Цель работы

изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнить программную реализацию методов

2. Порядок выполнения работы

1 часть. Решение нелинейного уравнения

Задание:

1. Отделить корни заданного нелинейного уравнения графически (вид уравнения представлен в табл. 6)
2. Определить интервалы изоляции корней.
3. Уточнить корни нелинейного уравнения (см. табл. 6) с точностью $\varepsilon=10^{-2}$.
4. Используемые методы для уточнения каждого из 3-х корней многочлена представлены в таблице 7.
5. Вычисления оформить в виде таблиц (1-5), в зависимости от заданного метода. Для всех значений в таблице удержать **3 знака** после запятой.

   5.1 Для метода половинного деления заполнить таблицу 1.

   5.2 Для метода хорд заполнить таблицу 2.

   5.3 Для метода Ньютона заполнить таблицу 3.

   5.4 Для метода секущих заполнить таблицу 4.

   5.5 Для метода простой итерации заполнить таблицу 5. **Проверить условие сходимости метода на выбранном интервале.**
6. Заполненные таблицы отобразить в отчете.

2 часть. Решение системы нелинейных уравнений

Задание:

1. Отделить корни заданной системы нелинейных уравнений графически (вид системы представлен в табл. 8).
2. Используя указанный метод, решить систему нелинейных уравнений с точностью до 0,01.
3. Для метода простой итерации проверить условие сходимости метода.
4. Подробные вычисления привести в отчете.

2 Программная реализация задачи:

Для нелинейных уравнений:

1. Все численные методы (см. табл. 9) должны быть реализованы в виде отдельных подпрограмм/методов/классов.
2. Пользователь выбирает уравнение, корень/корни которого требуется вычислить (3-5 функций, в том числе и трансцендентные), из тех, которые предлагает программа.
3. Предусмотреть ввод исходных данных (границы интервала/начальное приближение к корню и погрешность вычисления) из файла или с клавиатуры по выбору конечного пользователя.
4. Выполнить верификацию исходных данных. Необходимо анализировать наличие корня на введенном интервале. Если на интервале несколько корней или они отсутствуют – выдавать соответствующее сообщение. Программа должна реагировать на некорректные введенные данные.
5. Для методов, требующих начальное приближение к корню (методы Ньютона, секущих, хорд с фиксированным концом, простой итерации), выбор начального приближения x_0 (a или b) вычислять в программе.
6. Для метода простой итерации проверять достаточное условие сходимости метода на введенном интервале.
7. Предусмотреть вывод результатов (найденный корень уравнения, значение функции в корне, число итераций) в файл или на экран по выбору конечного пользователя.
8. Организовать вывод графика функции, график должен полностью отображать весь исследуемый интервал.

Для систем нелинейных уравнений:

1. Пользователь выбирает предлагаемые программой системы двух нелинейных уравнений (2-3 системы).
2. Организовать вывод графика функций.
3. Начальные приближения ввести с клавиатуры.
4. Для метода простой итерации проверить достаточное условие сходимости.
5. Организовать вывод вектора неизвестных: x_1, x_2.
6. Организовать вывод количества итераций, за которое было найдено решение.
7. Организовать вывод вектора погрешностей: $|x_i^{(k)} - x_i^{(k-1)}|$
8. Проверить правильность решения системы нелинейных уравнений.

3. Рабочие формулы используемых методов.

Методы решения нелинейных уравнений

1. Метод половинного деления

рабочая формула: $x_i = \dfrac{a_i + b_i}{2}$

критерий окончания: $|a_n - b_n| \leq \varepsilon$

2. Метод Ньютона

рабочая формула: $x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$

критерий окончания: $|x_n - x_{n-1}| \leq \varepsilon$

3. Метод простой итерации

рабочая формула: $x_{i+1} = \varphi(x_i)$

критерий окончания: $|x_{k+1} - x_k| \leq \varepsilon$

4. Метод хорд

рабочая формула: $x_i = a_i - \frac{b_i - a_i}{f(b_i) - f(a_i)} f(a_i)$

критерий окончания: $|x_n - x_{n-1}| \leq \varepsilon$

5. Метод секущих

рабочая формула: $x_{i+1} = x_i - \frac{f(x_i) \cdot (x_i - x_{i-1})}{f(x_i) - f(x_{i-1})}$

критерий окончания: $|x_n - x_{n-1}| \leq \varepsilon$

Методы решения систем нелинейных уравнений

1. Метод простой итерации для систем

рабочая формула:

$$\begin{cases} x_1^{(k+1)} = \varphi_1(x_1^k, x_2^k, \ldots, x_n^k) \\ x_2^{(k+1)} = \varphi_2(x_1^k, x_2^k, \ldots, x_n^k) \\ \ldots \\ x_n^{(k+1)} = \varphi_n(x_1^k, x_2^k, \ldots, x_n^k) \end{cases}$$

критерий окончания: $\max\limits_{1<i<n} |x_i^{(k+1)} - x_i^k| \leq \varepsilon$

2. Метод Ньютона для систем

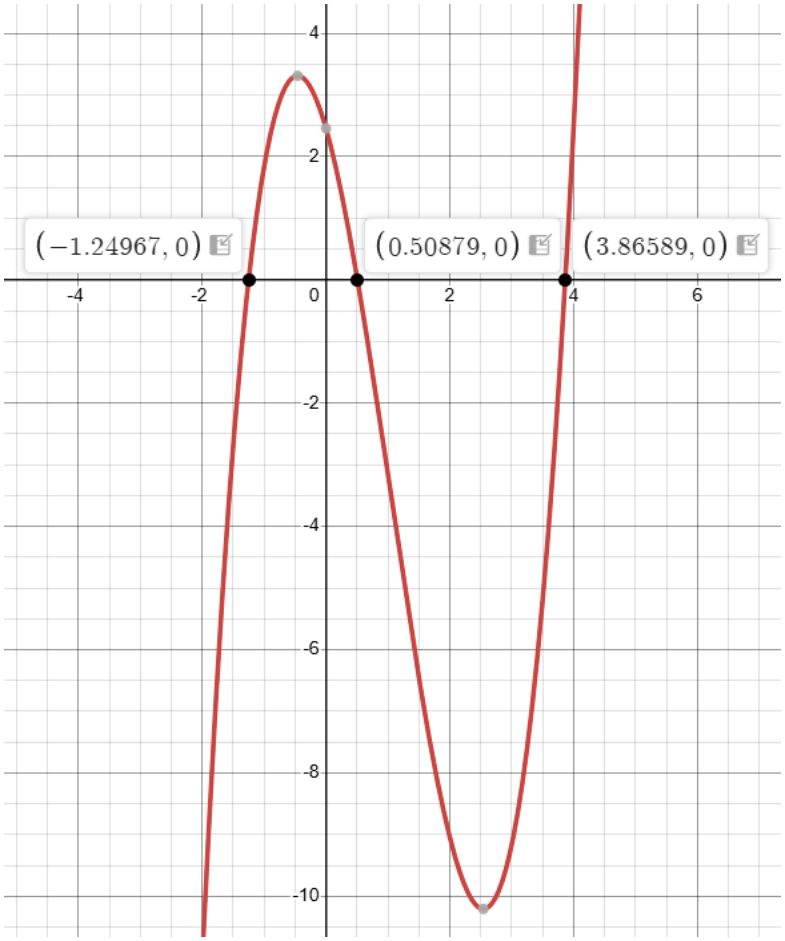
рабочая формула: $x_i + 1 = x_i + \Delta x_i$ и $y_i + 1 = y_i + \Delta y_i$

$$J(x_i, y_i) \cdot \begin{pmatrix} \Delta x_i \\ \Delta y_i \end{pmatrix} = - \begin{pmatrix} f_1(x_i, y_i) \\ f_2(x_i, y_i) \end{pmatrix}$$

$$J = \begin{pmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{pmatrix}$$

критерий окончания: $|x_{i+1} - x_i| \leq \varepsilon$ и $|y_{i+1} - y_i| \leq \varepsilon$

4. Графики функций на исследуемом интервале.



5. таблицы вычислительной части 1

Определим интервалы изоляции корней

Для определения интервалов изоляции корней данного уравнения, можно воспользоваться методом интервалов знакопеременности. Для этого нужно найти значения функции на различных интервалах и определить знак функции на каждом из них.

Получим приближенные значения корней:

x ≈ -1.25, x ≈ 0.5, x ≈ 3.87

Теперь нужно разбить ось x на 4 интервала: (-∞, -1.25), (-1.25, 0.5), (0.5, 3.87) и (3.87, +∞). На каждом из этих интервалов нужно определить знак функции.

Для этого можем вычислить значения функции в произвольной точке каждого интервала. Например, для интервала (-∞, -1.25) можно выбрать x = -2, для интервала (-1.25, 0.5) x = -1, для интервала (0.5, 3.87) x = 2, и для интервала (3.87, +∞) x = 4.

Для x = -2: f(-2) = -4.83
Для x = -1: f(-1) = 1.833
Для x = 2: f(2) = -16.63
Для x = 4: f(4) = 5.12

Знаки функции на каждом интервале будут соответственно:

(-∞, -1.25)	(-1.25, 0.5)	(0.5, 3.87)	(3.87, +∞)
-	+	-	+

Таким образом, мы получаем три интервала изоляции корней уравнения:

(-2, -1), (0, 1) и (3, 4)

Уточнение корня уравнения методом половинного деления

№ шага	a	b	x	f(a)	f(b)	f(x)	|a-b|
1	-2	-1	-1.5	-11.042	1.833	-2.698	1
2	-1.5	-1	-1.25	-2.698	1.833	-0.003	0.5
3	-1.25	-1	-1.125	-0.003	1.833	1.017	0.25
4	-1.25	-1.125	-1.188	-0.003	1.017	0.533	0.125
5	-1.25	-1.188	-1.219	-0.003	0.533	0.272	0.062
6	-1.25	-1.219	-1.234	-0.003	0.272	0.136	0.031
7	-1.25	-1.234	-1.242	-0.003	0.136	0.067	0.016
8	-1.25	-1.242	-1.246	-0.003	0.067	0.032	0.008

Уточнение корня уравнения методом Ньютона

№ шага	x_k	$f(x_k)$	$f'(x_k)$	x_{k+1}	$\lvert x_{k+1} - x_k \rvert$
1	4	2.458	19.5	3.874	0.126
2	3.874	0.139	17.31	3.866	0.008

Уточнение корня уравнения методом простой итерации

проверим условие сходимости

Преобразуем к виду x = φ(x) = x + λ * f(x)

На интервале [0; 1] производная f'(x) < 0. Максимальное по модулю значение f'(1) = -6.75

Выбираем λ = 0.2 (так как $-1 < 1 + \lambda f'(x) < 1 \Rightarrow 0 < \lambda < \frac{2}{6.75} \approx 0.29$).

Тогда φ'(x) = 1 + 0.2 * (3x^2 - 6.25x - 3.5) = 0.3 + 0.6x^2 - 1.25x

Проверяем граничные точки интервала:

|φ'(0)| = |0.3| = 0.3 < 1

|φ'(1)| = |-0.35| = 0.35 < 1

Условие сходимости |φ'(x)| ≤ 0.35 < 1 выполняется

№ шага	x_k	x_{k+1}	$f(x_{k+1})$	$\lvert x_{k+1} - x_k \rvert$
1	0.5	0.51	-0.009	0.01
2	0.51	0.509	0.002	0.002

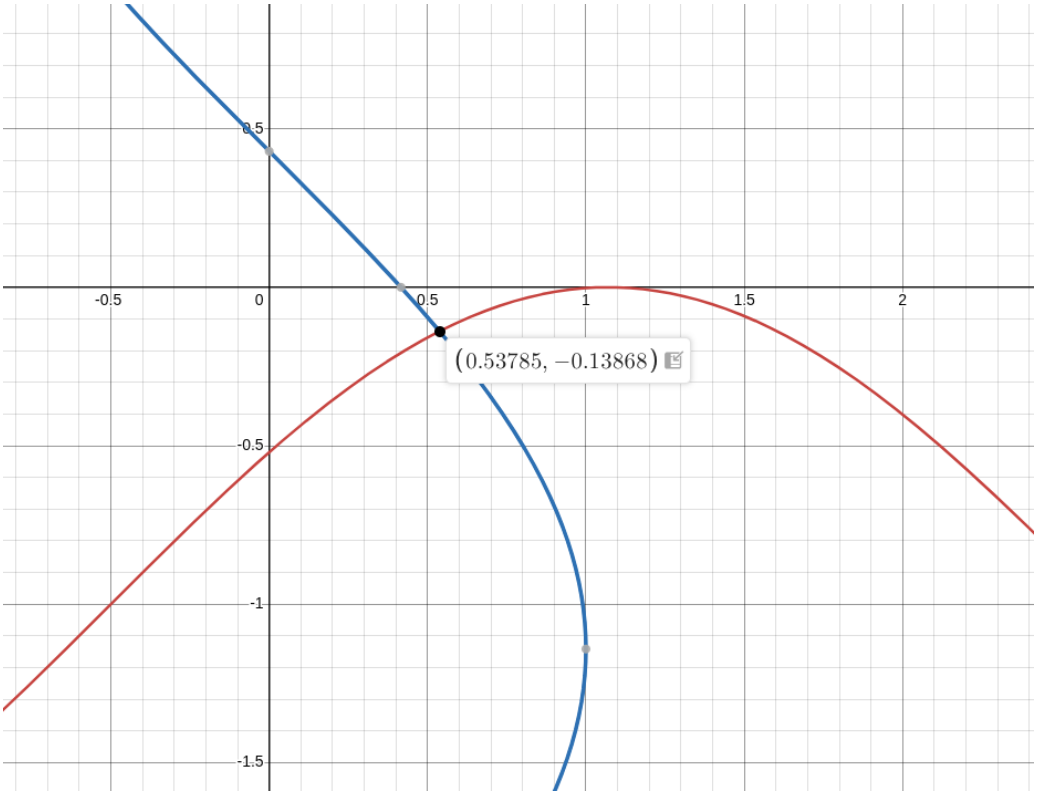
6. Подробное решение системы нелинейных уравнений

$\begin{cases} \sin(x + 0.5) - y = 1 \\ \cos(y - 2) + x = 0 \end{cases}$, Метод простой итерации (МПИ).

Определяем, что положительное решение системы уравнений находится в квадрате:

0 < x < 1

-1 < y < 0



Выразим переменные для итераций:

$$\begin{cases} x = -\cos(y-2) = \varphi_1(x,y) \\ y = \sin(x+0,5) - 1 = \varphi_2(x,y) \end{cases}$$

Проверим достаточное условие сходимости

$$|\frac{\partial \varphi_1}{\partial y}| + 0 = |\sin(y-2)| \approx |\sin(-2,1)| \approx 0,86 < 1$$

$$0 + |\frac{\partial \varphi_2}{\partial x}| = |\cos(x+0,5)| \approx |\cos(1,2)| \approx 0,36 < 1$$

$\max\limits_{x \in G} |\varphi'(x)| \le 0{,}86 < 1 \Rightarrow$ условие сходимости выполняется

$$\begin{cases} x = -\cos(y-2) \\ y = \sin(x+0,5) - 1 \end{cases}$$

Выберем начальное приближение: $x^{(0)}=0.7$, $y^{(0)}=-0.1$

1 шаг.

$x^{(1)}=-\cos(-0.1-2) = 0.505$ $\quad |x^{(1)} - x^{(0)}| = 0.195 > \varepsilon$

$y^{(1)} = \sin(0.7 + 0.5)-1=-0.068$ $\quad |y^{(1)} - y^{(0)}| = 0.032 > \varepsilon$

2 шаг.

$x^{(2)} =-\cos(-0.068 - 2) = 0.477$ $\quad |x^{(2)}-x^{(1)}|=0.028>\varepsilon$

$y^{(2)}=\sin(0.505 + 0.5) -1=-0.156$ $\quad |y^{(2)}-y^{(1)}|=0.088>\varepsilon$

3 шаг.

$x^{(3)} =-\cos(-0.156-2) =0.552$ $\quad |x^{(3)}-x^{(2)}|=0.075>\varepsilon$

$y^{(3)}=\sin(0.477 + 0.5) -1=-0.171$ $\quad |y^{(3)}-y^{(2)}|=0.015>\varepsilon$

4 шаг.

$x^{(4)} = -\cos(-0.171-2) = 0.565$ $\quad |x^{(4)} - x^{(3)}| = 0.013 > \varepsilon$

$y^{(4)} = \sin(0.552 + 0.5) - 1 = -0.131$ $\quad |y^{(4)} - y^{(3)}| = 0.04 > \varepsilon$

5 шаг.

$x^{(5)} = -\cos(-0.131-2) = 0.565$ $\quad |x^{(5)} - x^{(4)}| = 0.033 > \varepsilon$

$y^{(5)} = \sin(0.565 + 0.5) - 1 = -0.125$ $\quad |y^{(5)} - y^{(4)}| = 0.006 <= \varepsilon$

6 шаг.

$x^{(6)} = -\cos(-0.125-2) = 0.526$ $\quad |x^{(6)} - x^{(5)}| = 0.005 <= \varepsilon$

$y^{(6)} = \sin(0.532 + 0.5) - 1 = -0.142$ $\quad |y^{(6)} - y^{(5)}| = 0.017 > \varepsilon$

7 шаг.

$x^{(7)} = -\cos(-0.142-2) = 0.54$ $\quad |x^{(7)} - x^{(6)}| = 0.014 > \varepsilon$

$y^{(7)} = \sin(0.526 + 0.5) - 1 = -0.145$ $\quad |y^{(7)} - y^{(6)}| = 0.003 <= \varepsilon$

8 шаг.

$x^{(8)} = -\cos(-0.145-2) = 0.543$ $\quad |x^{(8)} - x^{(7)}| = 0.002 <= \varepsilon$

$y^{(8)} = \sin(0.54 + 0.5) - 1 = -0.137$ $\quad |y^{(8)} - y^{(7)}| = 0.007 <= \varepsilon$

7. Листинг программы

```
class SimpleIterationMethod(Method):
    def solve(self, eq, **kwargs) -> ResultMethod:
        x0 = kwargs.get('x0')
        eps = kwargs.get('eps', 0.0001)
        max_iter = kwargs.get('max_iter', 100)

        try:
            q = abs(eq.dphi(x0))
            if q >= 1:
                return ResultMethod(
                    success=False,
                    error_message=f"Условие сходимости |phi'(x)| < 1 не выполнено
(q={q:.4f}). Попробуйте другое начальное приближение."
                )
        except Exception as e:
            return ResultMethod(success=False, error_message=f"Ошибка при
вычислении производной phi: {e}")

        x = float(x0)
        history = []

        for i in range(1, max_iter + 1):
            try:
                x_next = eq.phi(x)

                err = abs(x_next - x)

                history.append([i, x_next, eq.f(x_next), err])
```

```
            if err < eps:
                return ResultMethod(
                    solutions=[x_next],
                    iterations_data=history,
                    errors=[err],
                    success=True
                )

            x = x_next

            if abs(x) > 1e10:
                return ResultMethod(success=False, error_message="Метод
расходится (значения x слишком велики)")

        except Exception as e:
            return ResultMethod(success=False, error_message=f"Ошибка
вычислений: {e}")

    return ResultMethod(
        success=False,
        error_message=f"Превышено максимальное число итераций
({max_iter})"
    )
```

```
class SecantMethod(Method):
    def solve(self, eq, **kwargs) -> ResultMethod:
        a = kwargs.get('a')
        b = kwargs.get('b')
        eps = kwargs.get('eps', 1e-4)

        if eq.f(a) * eq.ddf(a) > 0:
            x0 = a
            x1 = b
        else:
            x0 = b
            x1 = a

        history = []
        for i in range(1, 101):
            f0 = eq.f(x0)
            f1 = eq.f(x1)

            if abs(f1 - f0) < 1e-12:
                return ResultMethod(success=False, error_message="Деление на
ноль: разность функций близка к 0")

            x_next = x1 - f1 * (x1 - x0) / (f1 - f0)
            err = abs(x_next - x1)

            history.append([i, x_next, eq.f(x_next), err])

            if err < eps:
                return ResultMethod(solutions=[x_next], iterations_data=history,
errors=[err])
```

```
            x0 = x1
            x1 = x_next

        return ResultMethod(success=False, error_message="Метод секущих не
сошелся за 100 итераций")
```

```
class ChordMethod(Method):
    def solve(self, eq, **kwargs) -> ResultMethod:
        a = kwargs.get('a')
        b = kwargs.get('b')
        eps = kwargs.get('eps', 1e-4)
        max_iter = kwargs.get('max_iter', 100)

        if eq.f(a) * eq.ddf(a) > 0:
            c = a
            x = b
        else:
            c = b
            x = a

        history = []
        for i in range(1, max_iter+1):
            fx = eq.f(x)
            fc = eq.f(c)

            x_next = x - fx * (x - c) / (fx - fc)
            err = abs(x_next - x)

            history.append([i, x_next, eq.f(x_next), err])

            if err < eps:
                return ResultMethod(solutions=[x_next], iterations_data=history,
errors=[err])

            x = x_next

        return ResultMethod(success=False, error_message=f"Метод хорд не
сошелся за {max_iter} итераций")
```

```
class NewtonMethod(Method):
    def solve(self, system: NonLinearSystem, **kwargs) -> ResultMethod:
        x0 = kwargs.get('x0')
        eps = kwargs.get('eps', 1e-4)
        # Добавлено изменение максимального количества итераций
        max_iter = kwargs.get('max_iter', 100)

        # Извлекаем начальные приближения
        x, y = x0[0], x0[1]
        history = []

        for i in range(1, max_iter + 1):
            f_vals = system.evaluate([x, y])
            jac = system.jacobian([x, y])
```

```
        a, b = jac[0][0], jac[0][1]
        c, d = jac[1][0], jac[1][1]

        det = a * d - b * c
        if abs(det) < 1e-12:
            return ResultMethod(success=False, error_message="Определитель
Якобиана близок к нулю")

        dx = (-f_vals[0] * d - (-f_vals[1] * b)) / det
        dy = (a * (-f_vals[1]) - (c * (-f_vals[0]))) / det

        x += dx
        y += dy
        err = max(abs(dx), abs(dy))

        history.append([i, x, y, err])

        if err < eps:
            return ResultMethod(
                solutions=[x, y],
                iterations_data=history,
                errors=[abs(dx), abs(dy)],
                success=True
            )

    return ResultMethod(
        success=False,
        error_message=f"Метод не сошелся за {max_iter} итераций"
    )
```

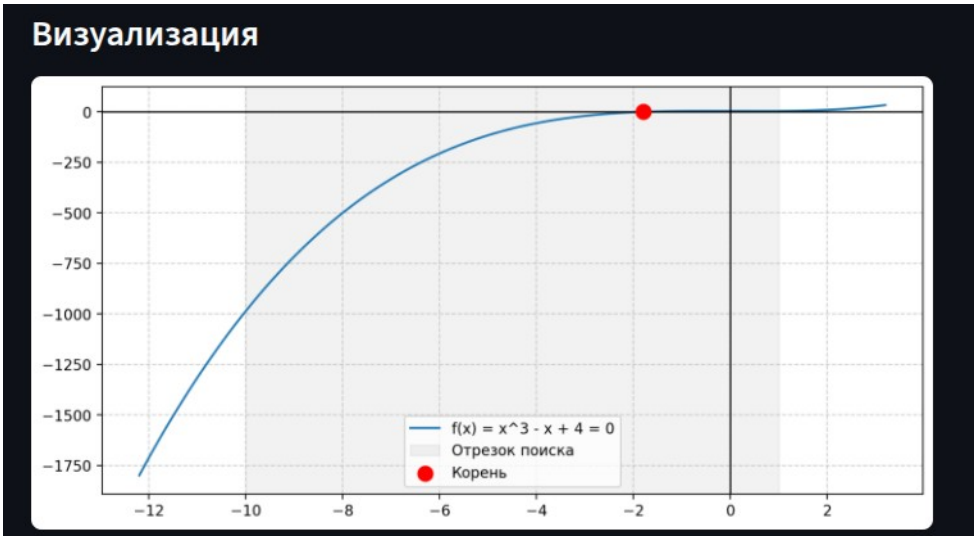
8. Результаты выполнения программы

Корень найден: **-1.796329**

Итераций	f(x)	Погрешность
6	-6.04e-05	6.04e-05

Таблица итераций

	№	x_n	f(x_n)	Погрешность
0	1	-2.4101	-7.5899	7.5899
1	2	-1.8576	-0.5525	0.5525
2	3	-1.8026	-0.055	0.055
3	4	-1.797	-0.0057	0.0057
4	5	-1.7964	-0.0006	0.0006
5	6	-1.7963	-0.00006	0.00006



9. Выводы по работе.

за время выполнения лабораторной работы я реализовал и изучил несколько методов для решений нелинейных уравнений и систем нелинейных уравнений